PHASE 6 OF 8

Project Testing

Verifying each module and the complete workflow

Project: LegalEase – AI-Powered Legal Document Generator

Presented by: Ganesh Prabhu, Gopinath, Hariharan, Gowtham

6.1 Purpose of this Phase

The Project Testing phase verifies that every module and the complete LegalEase workflow behave as required. Testing checks that valid inputs produce a draft, invalid inputs are rejected, edits are kept, and all three export formats work. Each test is linked to a requirement from the Requirement Analysis phase.

6.2 Test Strategy

- **Module testing:** the API, the AI generation, the interface and each formatter are tested separately.
- **Integration testing:** checks that the frontend, backend and Gemini work together.
- **End-to-end testing:** follows the full user journey from input to download.
- **Manual verification:** exported files are opened to confirm content and layout.

6.3 Test Environment

Item	Details
Platform	Local machine running Python with the project dependencies installed
Backend	FastAPI served by Uvicorn
Frontend	Streamlit application in a web browser
AI service	Google Gemini using a valid API key from the .env file
Viewers for output	A text editor, Microsoft Word (or compatible) and a PDF reader

6.4 Test Cases

#	Test	Verification	Expected result
1	Input test	Document type, parties, terms and date are accepted	All fields accept data without error
2	Generation test	A valid request returns generated content	Structured document text is produced

#	Test	Verification	Expected result
3	Preview test	Generated content is displayed	Full text appears correctly in the preview
4	Edit test	Edited content is retained	Changes remain and are used for export
5	TXT export test	A TXT file is produced	Readable text file downloads
6	DOCX export test	A formatted Word file is produced	Word file opens with headings and terms
7	PDF export test	A formatted PDF is produced	PDF opens with correct layout
8	API test	/generate validates input and returns content	Invalid input rejected; valid input returns text
9	End-to-end test	Complete frontend-to-AI-to-download workflow	User can go from input to downloaded file

6.5 Detailed Test Scenarios

Generation test. Enter a Lease Agreement with two parties, a set of terms, an effective date and a jurisdiction, then generate. The output should mention the parties, reflect the supplied terms and follow a document-like structure.

Edit test. Modify a sentence in the preview, then download the DOCX. The downloaded file should contain the modified sentence, not the original.

API test. Send a request with a required field missing; the API should return a validation error. Send a complete request; the API should return generated text.

6.6 Additional Checks

- Behaviour when a required field is left empty.
- Behaviour when the Gemini key is missing or invalid, or the network is unavailable.
- Long terms text and special characters in party names.
- Generating a second document after the first, to ensure old content is replaced.
- Opening each exported file on a different viewer to confirm compatibility.

6.7 Defect Handling

Any failure is recorded with the test number, the input used, the observed result and the expected result. The responsible team member fixes the issue, and the failed test is

repeated together with related tests to make sure nothing else was affected.

6.8 Exit Criteria

Testing is complete when all nine planned tests have been executed, defects that block the workflow are fixed, and the end-to-end test succeeds for each of the three export formats.

Result columns are intentionally not pre-filled: record the actual Pass/Fail result and date when each test is executed.

6.9 Requirement-to-Test Traceability

Requirement	Verified by test(s)
FR-1 to FR-4 (inputs)	1 - Input test
FR-5 (generate content)	2 - Generation test, 8 - API test
FR-6 (preview and edit)	3 - Preview test, 4 - Edit test
FR-7 (download)	5, 6 and 7 - TXT, DOCX and PDF export tests
Whole workflow	9 - End-to-end test

6.10 Test Data Guidelines

- Use realistic but non-confidential names and terms.
- Include at least one short input and one long, detailed input.
- Test several document types, such as Lease Agreement, NDA and Employment Contract.
- Repeat export tests after editing the text to confirm edits are exported.

6.11 Test Report Template

Field	Entry
Test number and name	e.g. 6 - DOCX export test
Input used	Document type, parties, terms, date, jurisdiction
Expected result	As listed in the test case table
Actual result	Recorded during execution
Status and date	Pass / Fail and date executed

6.12 Test Coverage by Component

Component	What is checked	Related tests
Streamlit frontend	Inputs accepted, preview shown, edits kept, buttons work	1, 3, 4
FastAPI backend	Route exists, validation works, response returned	2, 8
Gemini AI core	Prompt built from inputs, text returned	2, 9
TXT formatter	File created with correct content	5
DOCX formatter	Word file created with structure and branding	6
PDF formatter	PDF file created with structure and branding	7

6.13 Risks During Testing

- AI output can differ slightly between runs, so tests check for structure and inclusion of the supplied details rather than identical wording.
- Testing depends on the availability of the Gemini service and a valid key.
- Formatting must be checked by opening files, since a file can be created but still look wrong.

6.14 Sign-off

When the exit criteria are met, the team records the final test status and confirms that the application is ready for documentation and demonstration.